

Chapter # 4:-

Creating User Interface

- o It is vital to create compelling and intuitive user interfaces for applications.

Fundamental UI Design

xmlns:

- o it is namespace URI.
- o URI stands for uniform resource indicators.
- o In computing, it is a string of characters used to identify a name of resource.

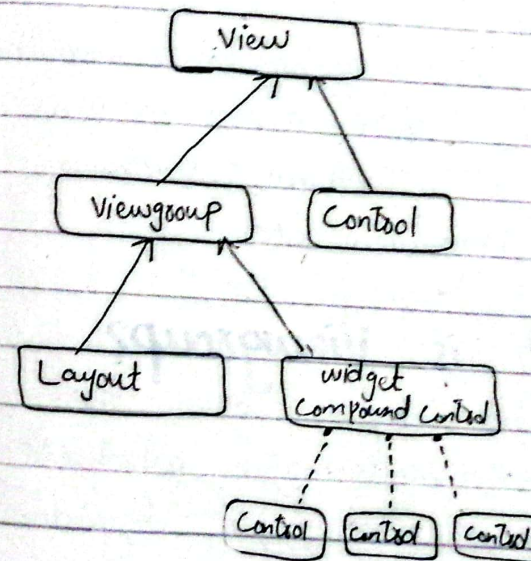
o xmlns: Prefix = "URI"

e.g <http://schemas.android.com/apk/res/android:id>

- o xml namespaces are used for providing uniquely named elements and attributes in a xml document.

Important Attributes:

- o android:id
- o android:layout_width (required)
- o android:layout_height (required)
- o android:text



Q. What is view?

⇒ View refers to the "android.view.View" class which is the base class of all the UI classes.

⇒ Views are the basic visual interface class for visual interface elements (commonly known as controls or widgets).

⇒ All uses interface controls, and the layout classes, are derived from Views.

⇒ it just like a rectangular box which can response on user action.
e.g. Some child classes of View class are TextView, Button, checkBox, ImageView etc.

Q. What is ViewGroup?

⇒ ViewGroup refers to the "android.view.ViewGroup" which is the parent class for layouts and layouts parameters.

⇒ ViewGroup is the invisible container which holds:

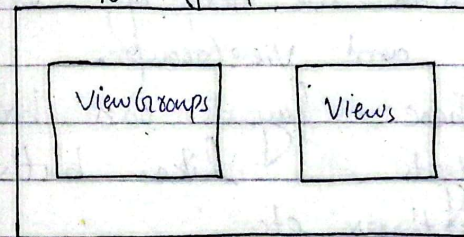
- Views
- ViewGroups.

⇒ ViewGroups are extensions of the View class that contain multiple child Views.

⇒ By extending the ViewGroup class, Compound Controls can be made up of interconnected child Views.

⇒ The ViewGroup class is also extended to provide the layout managers such as LinearLayout.

View group



⇒ ViewGroup is a super class of layouts.

Q. What is layout? Explain different types of layouts in android?

Layout:

Layout refers to the arrangement of element.

⇒ They are part of Android Jetpack.

⇒ They define the structure of android user interface in the app, like in an activity.

⇒ All elements in the layout are built with the help of Views hierarchy and ViewGroups.

⇒ These layouts can have various widgets like buttons, labels, textboxes etc.

Attributes of layout:

The following are the attributes for customizing a layout.

android:id:

◦ it uniquely identifies the android layout.

android:hint:

◦ it shows the hint of what to fill inside the EditText.

android:layout_height:

◦ It sets the height of the layout.

android:layout_width:

◦ it sets the width of the layout.

android:layout_gravity:

◦ it sets the position of the child view.

android:layout_marginTop:

◦ it sets the margin of the from the top of the layout.

android:layout_marginBottom:

◦ Sets from the bottom of the layout.

android:layout_marginLeft:
• sets from the left of the layout.

android:layout_marginRight:
• sets from the right of the layout.

android:layout-x:
• it specifies the x coordinates of the layout.

android:layout-y:
• it specifies the y coordinates of the layout.

How to declare layout?

⇒ The layout declare in two ways:

• Declare UI elements in XML:-

⇒ Android provides a straight-forward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts.

⇒ Layout editor use to build xml layout using a drag-and-drop interface.

2. Instantiate layout elements at runtime:-

⇒ App can create View and ViewGroup objects programmatically.

Layout parameters:

⇒ XML layout attributes named layout-something define layout parameters for the View that are appropriate for the ViewGroup in which it resides.

ViewGroup.layoutparams.

• wrap-Content:

- tells the view to size itself to the dimensions required by its content.

o match-parent:

tells view to become as big as its parent view group will allow.

the available space: linear weights

Layout Types:-

(1) LinearLayout:

⇒ A layout that organizes its children into a single horizontal or vertical row. Aligns all children in single direction.

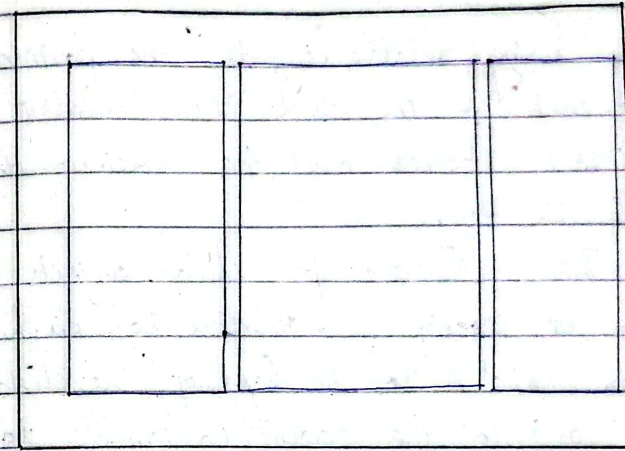
⇒ it creates a scrollbar if the length of the window exceeds the length of the screen.

⇒ A LinearLayout adds each child view in a straight line.

⇒ A vertical layout has one child view per row.

⇒ While a horizontal layout has a single row of views.

⇒ The LinearLayout manager allows to specify a "weight" for each child view that controls the relative size of each within

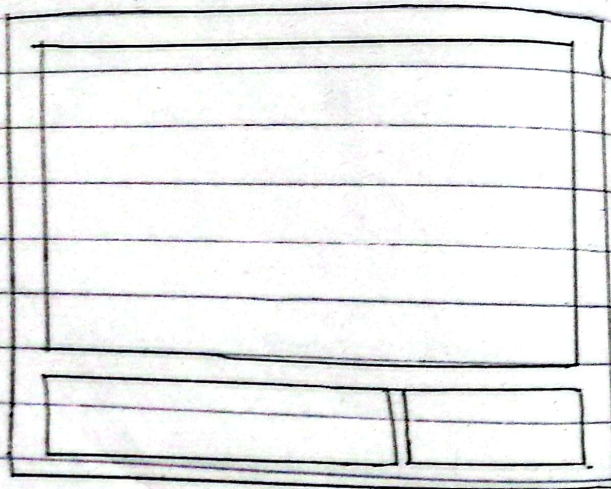


(a) LinearLayout

2. Relative Layout:-

⇒ By using relative layouts, the position of each of the child Views relative to each other and the screen boundaries can be define.

⇒ The location of child objects can be specify relative to each other (child A to the left of the child B) or to the parent (aligned to the top of the parent).



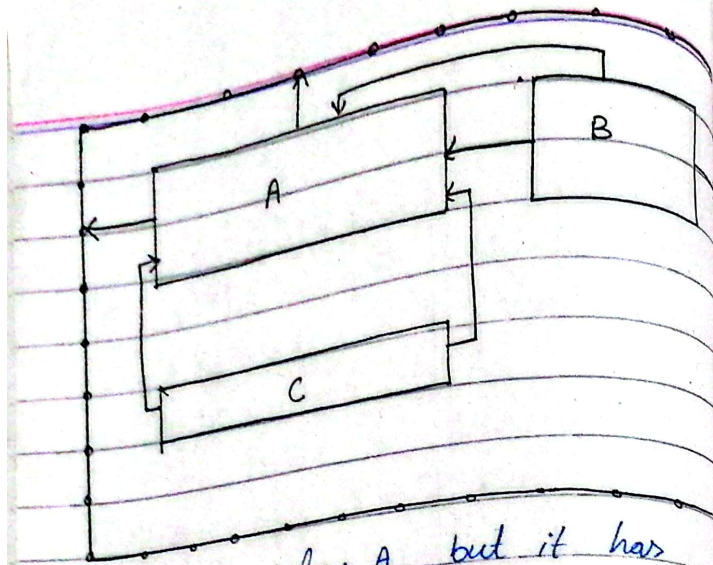
⇒ used for more complex and adaptable designs.

3. Constraint Layout:-

⇒ It offers a flexible way to create complex UIs by defining relationships (constraints) between elements, ensuring they adapt well to different screen size.

⇒ This layout is used to make our app dynamic and responsive.

⇒ It is similar to relative layout where views are setup according to the relationship b/w siblings view and parent layout, but it is more flexible.



View C below A, but it has no vertical constraint.

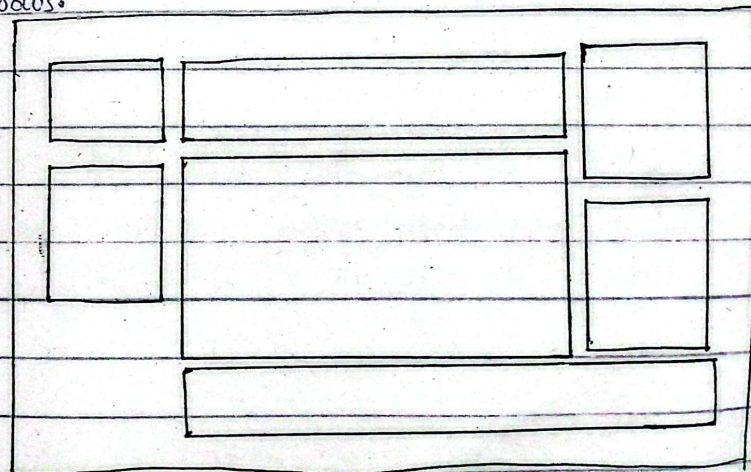
4 TableLayout:

⇒ TableLayout presents its children in rows and columns.

⇒ Tables can span multiple rows and columns, and columns can set to shrink or grow.

⇒ TableLayout containers do not display row, column, and cell border lines. The table will have the same number of columns as the row with the most cells.

⇒ A table can leave cells vacant. Here, cells cannot span multiple rows.

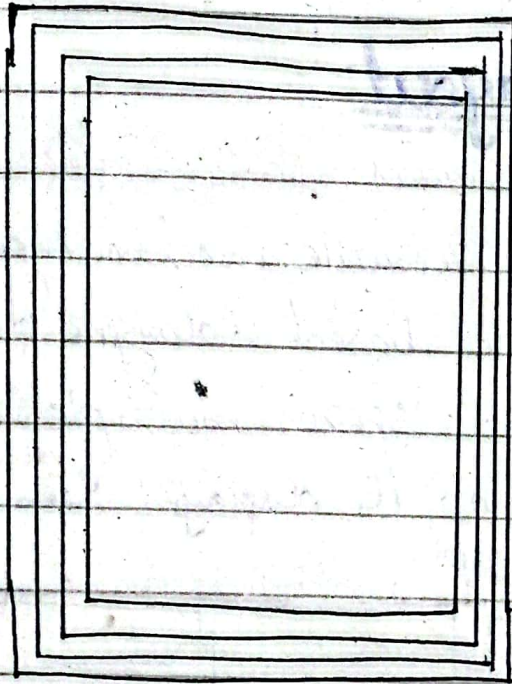


5. Frame Layout:-

⇒ The simplest of the layout managers, the Frame Layout simply pins each child view to the top left corner.

⇒ Adding multiple children stacks to each new child on top of the previous, with each ^{new} view obscuring the last.

⇒

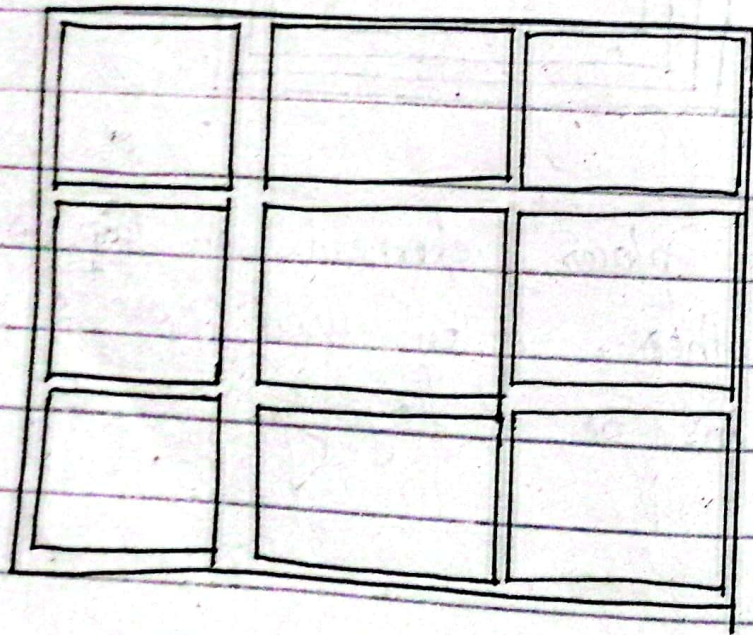


- it is a placeholder on screen that use to display a single view.

⇒ it places elements on top of each other, often used for simple animations or layering.

6. Grid Layout:

⇒ it is similar to Table Layout but more versatile, allowing to create grid-based designs with items that span multiple rows and columns. it displays items in 2D, scrollable grid.

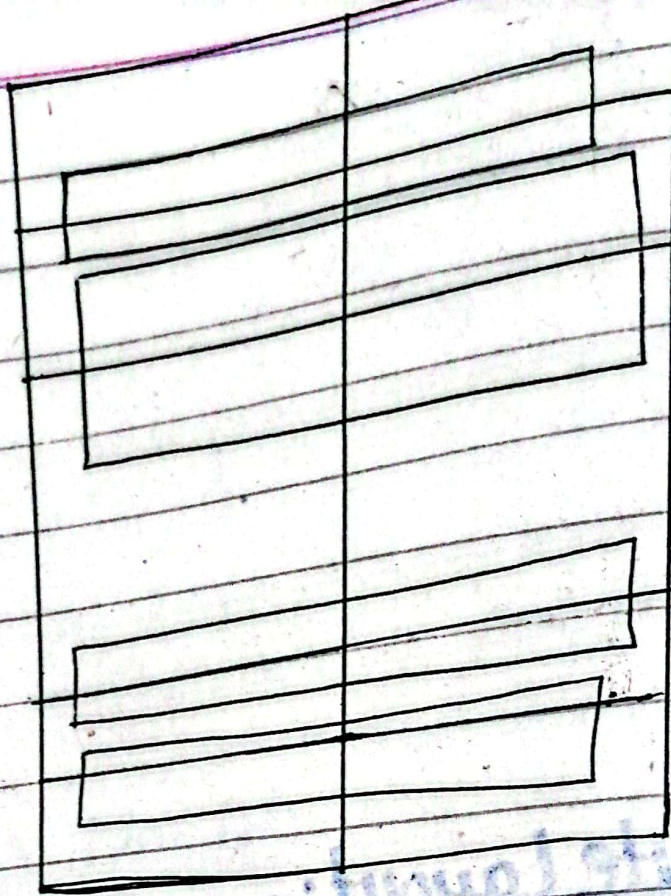


7. Absolute Layout:-

⇒ In absolute layout, each child Views position is defined in absolute coordinates.

⇒ it positions elements at specific X-Y coordinates, but it's less commonly used because it does not adapt well to various screen sizes.

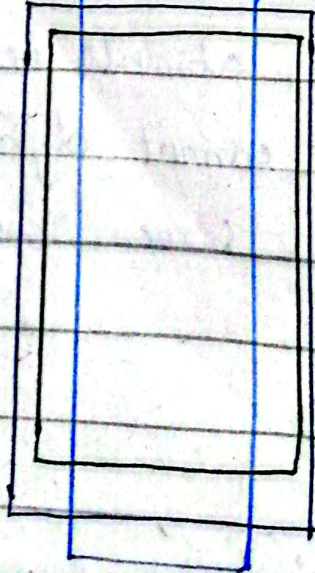
⇒ A layout in absolute terms means that layout cannot dynamically adjust for different screen resolutions and orientations.



Absolute layout

8. Scroll View:

A Container that allows for scrolling content, useful when the content screen exceeds the screen size.



Q. How to create Activity user interface with views?

⇒ A new Activity starts with an empty screen onto which a user interface is placed.

⇒ call `setContentView`, passing in the View instance (typically a layout) to display.

⇒ The `setContentView` method accepts either a layout resource ID or a single view instance.

⇒ The following code sets the user interface for an activity using an external layout resource.

@Override

```
public void onCreate(Bundle savedInstanceState) {
```

```
    super.onCreate(savedInstanceState);
```

```
    setContentView(R.layout.main);
```

```
    TextView myTextView = (TextView) findViewById(R.id.myTextView);
```

```
}
```